

Praxis: Einen einfachen Character-Device-Treiber bauen

Ein vollständiges Labor von char-dev-demo.c bis zum reproduzierbaren Funktionstest

3.11

Das Labor liefert einen vollständigen Lifecycle

1	Vorbereiten	Testsystem und passende Kernel-Header prüfen
2	Implementieren	Zustand, Callbacks und Registrierung schreiben
3	Bauen	externes Modul mit Kbuild erzeugen
4	Ausführen	Modul laden und Geräteknoten beobachten
5	Prüfen	read/write, Grenzen und Rückbau testen

Definition of Done: Write und anschließendes Read liefern dieselben akzeptierten Bytes.

Ein isoliertes Testsystem begrenzt das Risiko

01 Laufender Kernel

uname -r notieren; Modul muss zu diesem Kernel passen.

02 Build-Verzeichnis

/lib/modules/\$(uname -r)/build muss verfügbar sein.

03 Werkzeuge

Compiler, make und passende Kernel-Header installieren.

04 Sicherer Rahmen

VM, Snapshot oder entbehrliches Testgerät verwenden.

```
$ uname -r
$ test -d /lib/modules/$(uname -r)/build && echo READY
```

Kernelmodule laufen mit höchsten Rechten: Tippfehler können das System blockieren oder zum Absturz bringen.

Zwei Dateien reichen für das externe Modul

char-dev-demo/

└─ char-dev-demo.c
└─ Makefile

Quellcode und Kbuild-Anweisung
bleiben in einem Arbeitsverzeichnis.

Makefile

```
obj-m += char-dev-demo.o

KDIR ?= /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(CURDIR) modules

clean:
    $(MAKE) -C $(KDIR) M=$(CURDIR) clean
```

M=\$(CURDIR) kennzeichnet das Verzeichnis als externes Modul; Kbuild übernimmt Kernel-Flags und Abhängigkeiten.

Der Gerätezustand kapselt alle veränderlichen Daten

```
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/mutex.h>
#include <linux/uaccess.h>

#define LAB_NAME "char-dev-demo"
#define LAB_SIZE 128
```

```
struct lab_dev {
    struct cdev cdev;
    struct mutex lock;
    char data[LAB_SIZE];
    size_t len;
};

static dev_t lab_devt;
static struct class *lab_class;
static struct lab_dev lab;
```

cdev, Lock und Buffer haben denselben Besitzer und damit einen nachvollziehbaren Lifecycle.

file_operations legt den unterstützten Vertrag fest

```
static int lab_open(struct inode *, struct file *);
static int lab_release(struct inode *, struct file *);
static ssize_t lab_read(struct file *, char __user *, size_t, loff_t *);
static ssize_t lab_write(struct file *, const char __user *, size_t, loff_t *);

static const struct file_operations lab_fops = {
    .owner    = THIS_MODULE,
    .open     = lab_open,
    .release  = lab_release,
    .read     = lab_read,
    .write    = lab_write,
    .llseek   = no_llseek,
};
```

- 01 owner
Modul referenzieren
- 02 open
Instanz in private_data ablegen
- 03 read/write
gepufferten Datentransfer ausführen
- 04 no_llseek
keine frei wählbare Position anbieten

Nur tatsächlich implementierte Semantik wird veröffentlicht.

open löst die Instanz genau einmal auf

```
static int lab_open(struct inode *inode, struct file *file)
{
    struct lab_dev *d;
    d = container_of(inode->i_cdev, struct lab_dev, cdev);
    file->private_data = d;
    return 0;
}

static int lab_release(struct inode *inode, struct file *file)
{
    return 0;
}
```

01 container_of

Vom eingebetteten cdev zur umschließenden Geräteinstanz.

02 private_data

Alle Folgezugriffe erhalten die Instanz ohne globale Suche.

03 release

Kein Open-Zustand vorhanden – deshalb bleibt der Callback leer.

read arbeitet mit einem konsistenten Snapshot

```
static ssize_t lab_read(struct file *file, char __user *buf,
                        size_t count, loff_t *ppos)
{
    struct lab_dev *d = file->private_data;
    char tmp[LAB_SIZE];
    size_t len;

    mutex_lock(&d->lock);
    len = d->len;
    memcpy(tmp, d->data, len);
    mutex_unlock(&d->lock);

    return simple_read_from_buffer(buf, count, ppos, tmp, len);
}
```

- 01 Sperren
gemeinsamen Zustand stabilisieren
- 02 Kopieren
kurzen Kernel-Snapshot bilden
- 03 Entsperren
Lock vor Usercopy wieder freigeben
- 04 Ausgeben
Position und EOF durch Helper behandeln

Der Kernel-Snapshot trennt Locking und Usercopy.

write übernimmt Daten erst nach erfolgreicher Usercopy

```
static ssize_t lab_write(struct file *file,
                        const char __user *buf, size_t count, loff_t *ppos)
{
    struct lab_dev *d = file->private_data;
    char tmp[LAB_SIZE];
    size_t n = min(count, sizeof(tmp) - 1);

    if (copy_from_user(tmp, buf, n))
        return -EFAULT;
    tmp[n] = '\0';

    mutex_lock(&d->lock);
    memcpy(d->data, tmp, n + 1);
    d->len = n;
    mutex_unlock(&d->lock);
    return n;
}
```

01 Begrenzen

Maximal 127 Nutzbytes akzeptieren.

02 Prüfen

Fehlerhafte Useradresse liefert -EFAULT.

03 Commit

Shared State nur unter dem Mutex ersetzen.

Bei zu großer Eingabe signalisiert der positive Rückgabewert bewusst einen Partial Write.

init veröffentlicht das Gerät erst nach Vorbereitung

```
static int __init lab_init(void)
{
    struct device *device;
    int ret;

    mutex_init(&lab.lock);
    ret = alloc_chrdev_region(&lab_devt, 0, 1, LAB_NAME);
    if (ret) return ret;

    cdev_init(&lab.cdev, &lab_fops);
    ret = cdev_add(&lab.cdev, lab_devt, 1);
    if (ret) goto err_region;

    lab_class = class_create(LAB_NAME);
    if (IS_ERR(lab_class)) {
        ret = PTR_ERR(lab_class);
        goto err_cdev;
    }
    device = device_create(lab_class, NULL, lab_devt,
                          &lab, "char-dev-demo0");
    if (IS_ERR(device)) {
        ret = PTR_ERR(device);
        goto err_class;
    }
    return 0;
...
}
```

- 01 Zustand
Mutex vor Sichtbarkeit
- 02 Nummer
dynamisch reservieren
- 03 cdev
Callbacks live schalten
- 04 Device
Userspace benachrichtigen

Nach `cdev_add()` können Callbacks laufen – der gemeinsame Zustand muss dann vollständig gültig sein.

Fehlerpfade und exit bilden den exakten Rückweg

```
...  
  
err_class:  
    class_destroy(lab_class);  
err_cdev:  
    cdev_del(&lab.cdev);  
err_region:  
    unregister_chrdev_region(lab_devt, 1);  
    return ret;  
}  
  
static void __exit lab_exit(void)  
{  
    device_destroy(lab_class, lab_devt);  
    class_destroy(lab_class);  
    cdev_del(&lab.cdev);  
    unregister_chrdev_region(lab_devt, 1);  
}  
  
module_init(lab_init);  
module_exit(lab_exit);  
MODULE_LICENSE("GPL");
```

Erwerb

Rückbau

dev_t

→ zuletzt

cdev

→ danach

class

→ danach

device

→ zuerst

Kbuild erzeugt und beschreibt das Modul

```
$ make  
  
$ modinfo ./char-dev-demo.ko  
$ file ./char-dev-demo.ko
```

- 01 char-dev-demo.o
kompiliertes Objekt
- 02 char-dev-demo.mod.*
Modulmetadaten
- 03 Module.symvers
exportierte Symbolversionen
- 04 char-dev-demo.ko
ladbares Kernelmodul

Ein erfolgreicher Build garantiert nur binäre Kompatibilität – nicht korrekte Laufzeitsemantik.

Laden und Beobachten prüfen die Registrierung

```
$ sudo insmod ./char-dev-demo.ko  
  
$ lsmod | grep char-dev-demo  
$ grep char-dev-demo /proc/devices  
$ ls -l /dev/char-dev-demo0  
  
$ sudo dmesg | tail -n 20
```

- 01 Modul
erscheint in lsmod
- 02 Major
steht in /proc/devices
- 03 Knoten
/dev/char-dev-demo0 trägt passende dev_t
- 04 Log
keine Warnung oder Oops

Der Geräteknoten kann kurz verzögert erscheinen, weil ihn typischerweise udev nach dem uevent anlegt.

Funktionstests prüfen Datenweg und Grenzen

```
$ printf 'hello' | sudo tee /dev/char-dev-demo0 > /dev/null
$ sudo cat /dev/char-dev-demo0
# Erwartung: hello

$ sudo sh -c "printf 'second value' > /dev/char-dev-demo0"
$ sudo cat /dev/char-dev-demo0
# Erwartung: second value

$ sudo rmmod char-dev-demo
$ test ! -e /dev/char-dev-demo0 && echo REMOVED
```

- 01 Happy Path
Schreiben → Lesen identisch
- 02 Overwrite
letzter Write ersetzt den Zustand
- 03 EOF
cat beendet sich nach dem ersten Datenblock
- 04 Cleanup
Geräteknoten verschwindet nach rmmod

Zusatztest: Mehr als 127 Bytes schreiben und den positiven Partial-Write-Rückgabewert beobachten.

Das Labor ist bestanden, wenn jede Invariante hält

- ✓ Build `char-dev-demo.ko` entsteht ohne Fehler
- ✓ Registrierung Major und `/dev/char-dev-demo0` sind konsistent
- ✓ Datenweg `read` liefert den letzten akzeptierten Write
- ✓ Grenze maximal 127 Nutzbytes werden übernommen
- ✓ Rückbau `rmmod` entfernt Device, Klasse, `cdev` und `dev_t`